

Uppgift 4 – NavMesh och Companion

Mål:

Målet är att lära sig vad en navmesh är och hur den kan användas för att navigera en AI. Vi ska också lära oss om tekniker kopplade till navmesh, som path following och path smoothing.

Generella krav:

- Det går bra att använda TGE, er egen motor eller ett tredje parts bibliotek (t.ex. raylib eller SDL).
- Programmet ska kompilera utan kompileringsvarningar på warning level 4 och med "treat warnings as errors" satt till "yes".
- Trådar som används ska joinas korrekt när programmet stängs av.
- Actors får inte flimra/hacka, deras movement ska vara smooth. (Inom rimliga gränser. Om en actor gör ett pyttelitet "hack" när den når sitt mål så är det ok, men den ska inte kontinuerligt bete sig glitchigt. Är du osäker om din movement är tillräckligt smooth så poke:a mig innan inlämning så gör jag en bedömning.)

Uppgift-specifika krav:

- Navmeshen som ni använder kan vara den som finns på Omniway, men det går även bra att läsa in en egen navmesh från Maya/Unity/Unreal. Det enda kravet är att navmeshen ska läsas in (den får inte vara hårdkodad) och den måste vara tillräckligt "intressant". Den får inte vara en kvadrat med två trianglar. Tänk på att navmeshen inte bör innehålla några andra polygoner än trianglar, annars kan ni få dubbeltrubbel med funnelling.

Uppgiften:

- Ladda in navmeshen på Omniway eller en egen navmesh. Det finns inladdningskod på Omniway om ni väljer att använda navmeshen som finns där. Det är valfritt om man vill använda den färdiga inladdningen eller om man vill koda sin egen.
- Rita ut alla noder. Det är rekommenderat men inte obligatoriskt att på något sätt kunna rita ut en nods connections, antingen via en checkbox i imgui eller genom att cykla genom alla noder t.ex. med space för att inspektera detaljerad nod-data.
- Varje triangel i navmeshen blir en nod i er path finding. Använd avståndet mellan trianglarna som g-värde.
- Det ska finnas två actors. En spelare och en companion.
- Spelaren styrs med ett arrive behavior och rör sig dit man senaste klickade på skärmen. Spelaren är inte begränsad av navmeshen.
- Spelarens companion ska röra sig på navmeshen och röra sig till den punkten på navmeshen som är närmast spelarens target. Är spelarens target på navmeshen ska dess companion röra sig dit. Är spelarens target utanför navmeshen ska dess companion röra sig så nära spelarens target den kan utan att lämna navmeshen.

- Vägen som spelarens companion tar ska inte gå mellan varje triangels mittpunkt utan ska istället vara smooth:ad med något post process pass, t.ex. funnelling. Det finns kod på Omniway för funnelling. Det är valfritt om man vill använda den färdiga funnelling eller koda sin egen.
- Ni ska rita ut både den o-smooth:ade och den smooth:ade pathen er companion tar.
- Spelarens companion kan med fördel styras med steering behavior som heter PathFollowing, men det är inget krav.
- Allt som allt ska användaren kunna styra runt spelaren med mus-klick var som helst på skärmen och ha en companion som följer efter spelaren men som är begränsad till att röra sig på navmeshen.

Inlämning:

- Denna gången är inlämningen en code review! När ni är klara, poke:a mig så kommer jag förbi och så kollar vi på resultatet tillsammans.
- När jag säger att er lösning är godkänd så behöver ni bara "lämna in" en tom inlämning på Omniway så godkänner jag er där.
- Som vanligt går det fint att parprogrammera.

Teknisk specifikation:

- En navmesh är ofta en fbx, som den på Omniway. Använd TGAs FBX importer för att ladda in navmeshen i er kod.
 - Importern finns i TGE, men ni kan också hämta den på p4 om ni väljer en annan motor för kursen
 - TGA/FBX/tgafbx-mainline om ni vill ha source
 - TGA/FBX/tgafbx-release om ni vill ha headers och lib
- En TGA::FBX::NavMesh innehåller Chunks, och varje Chunk en lista med vertices och en lista med polygoner. En polygon i detta sammanhanget är en lista med vertex indices.
- Navmeshen på Omniway är genererad från Unreal, så dess positioner är i cm och är i intervallet 0 till ca 5000.
 - Om ni använder min inladdning så ser ni att jag använder en funktion som heter GameplayUtils::ToWorldSpace. Den är till för att översätta normaliserade värden till gameplay-värden. Men om ni vill kan ni jobba i pixel-värden rakt av, dvs scale:a om 0-5000 till er resolution.

<fortsättning på nästa sida>

- Om ni använder min kod för inladdning och funnelling så kommer ni märka att där finns lite kod ni måste ersätta eller funktioner som ni måste implementera.
 - Ett par av mina egna containers, t.ex. `HD_GrowingArray` och `HD_String`. Använd `std::vector` och `std::string`, eller era egna.
 - `LoadFbx` innehåller `SetConnections`. Den loopar igenom alla noder och sätter deras `connections`-pekare.
 - `PerformFunnelling` innehåller `GetPortalBetweenNodes` och `GetVectorRelation`
 - `GetPortalBetweenNodes`
 - En portal är en struct som innehåller två positioner, `myLeft` och `myRight` (en portal har alltså en riktning). Den här funktionen tar först emot en `fromNode` och sen en `toNode` och returnerar ett portal-objekt som har sina positioner satta till rätt vertiser.
 - `GetVectorRelation`
 - Den här funktionen tar först emot `aForward` och sen `aVector`. Den returnerar ett enum: `VectorRelation::Left` om `aVector` är till vänster om `aForward`, `VectorRelation::Right` om den är till höger, och `VectorRelation::Aligned` om de är align:ade.